

Reproducibility Supplement for *Bilateral Deficiency*

Artifact status: separately implemented research artifact, unrefereed

Reproduction date: 9 August 2026

Platform used for the recorded run: macOS 26.5.2 (build 25F84), Python 3.14.6, Apple clang 21.0.0, Lean 4.32.1

Scope: indexed CNF semantics, exhaustive finite checks, formula-graph translation, structural analysis, algebraic conformance benchmarks, three native- and Lean-checked finite threshold encodings, a parser-independent Lean terminal-signature lemma, and a connected-family generator

Candidate identity: version 1.0.0-candidate, <https://doi.org/10.5281/zenodo.21857209>, source at <https://github.com/ipitchford/bilateral-deficiency>

S1. Assurance boundary

The artifact has six distinct evidence roles.

1. The manuscript's universal results are mathematical proofs. Passing code is not evidence that every formula satisfies a theorem.
2. Exhaustive receipts establish what the compiled program reported after enumerating the declared finite search space. A receipt is not a formal proof log and still depends on the implementation.
3. Cross-model tests compare independently expressed formula and graph semantics over a generated finite corpus. They are strong regression tests, not a proof for inputs outside the corpus.
4. LRAT certificates establish unsatisfiability of three regenerated finite SAT encodings. The written encoding lemma and clean-room conformance checks are the bridge from those CNFs to the bilateral-deficiency lower bounds.
5. A parser-independent Lean theorem proves the four-entry terminal signature directly from the ten hard-coded enforcer clauses. Its remaining translation boundary is the visible transcription of those clauses.
6. Compositional manifests apply the proved additivity theorem after checking component values, block structure, and deterministic graph reconstruction.

Lean's verified LRAT checker accepts each direct finite certificate. The package does not claim a fully formalized bilateral- deficiency theorem, a formally verified DIMACS parser or encoding compiler, global historical priority, peer review, or empirical superiority over other solvers.

S1.1 Imported theorem dependencies

Imported result	Exact theorem or object used	Convention transfer	Locally checked consequence
Zhang--Peitl--Szeider, SAT 2024, Appendix A.1	Displayed ten-clause $E_{3,2,2}$ enforcer and its forced-terminal property	Indexed clauses are transcribed explicitly; terminal polarity is oriented as in the manuscript; signed occurrences are recounted	Python enumeration and a parser-independent Lean theorem give exactly the four terminal rows used by the amplifier
Zhang--Peitl--Szeider, Theorem 15 and Table 1	Minimum 20 distinct clauses for an unsatisfiable $(3, 2, 2)$ -formula	Delete tautologies, collapse indexed duplicates, and read the remaining formula in the source's width-three, signed-occurrence-at-most-two convention	Positive β , incidence counting and graph order give Corollary 13's DIM-qualified order-50 bound
O--West, Corollaries 2.3 and 4.4 and Theorem 5.2	Connected cubic matching lower bound and infinite equality family H_1	Their n is the skeleton order r ; their α' is v	The edge-cover amplifier converts $r - v(R)$ into a construction-specific asymptotic gap density $1/72$

THEOREM_DEPENDENCIES.md gives the standalone ledger. Local interface checks do not independently re-prove the cited results.

S2. Contents

Path	Role
bd_core.py	Reference indexed-CNF semantics, residuals, bilaterality, exact enumeration, formula graph, inverse maps, algebraic operations, DIMACS I/O
bd_exact.cpp	Dependency-free exhaustive C++ solver with a deterministic JSON receipt
test_bd_core.py	Unit, exhaustive small-corpus, polynomial-spectrum, width-two, lift, replication, and regular-DIM tests
test_workflows.py	Generated graph binding and corruption-rejection tests
analyze_formula.py	Structural recognition, theorem-driven method recommendations, and regular-DIM gap interpretation
generate_benchmark.py	Prescribed-value algebraic conformance benchmark generator
verify_benchmark.py	Separate template solving, block reconstruction, hash checking, and additive-value verification
generate_positive_base_cnf.py	Deterministic encoder for the proposition $\beta(P) \leq 0$
verify_threshold_encoding_cleanroom.py	Standard-library clean-room clause reconstruction, independent DPLL semantic corpus, and mutation rejection
terminal_signature.py	Exhaustive finite terminal-cost and polarity-mask calculator
lean_terminal_signature.lean	Parser-independent Lean theorem for the complete four-entry enforcer signature
TERMINAL_CALCULUS.md	Universal min-plus composition and closing rules for terminal signatures
generate_enforcer_base.py	Composition of opposite-polarity published enforcers into a second positive base
generate_prism_amplifier.py	Connected exact signed-occurrence (3, 2, 2) prism/enforcer family generator
generate_owest_amplifier.py	O–West extremal skeleton and edge-enforcer amplifier generator
paired_edge_normal_form.py	Exact clause-multigraph representation and matching-restriction benchmark
search_paired_matching_conjecture.py	Deterministic falsification search for an invalid matching-only simplification
verify_positive_base_proof.py	Byte regeneration, direct witness check, native LRAT checks, Lean LRAT check, and deterministic receipt
verify_enforcer_composition_proof.py	Byte regeneration and native/Lean proof replay for the attributed second positive base
verify_connected_switch_proof.py	Reconstruction, witness check, byte regeneration, and native/Lean replay for the connected positive switch
lean_lrat_check.lean	Narrow DIMACS parser plus Lean core LRAT verification wrapper
proof/	Encoding, map, DRAT/LRAT certificates, proof transcripts, and the written encoding lemma
connectors/pp-beta1/	Extended optional connected-switch formula, encodings, map, receipt, and separately distributed 5.79 GB direct LRAT proof
third_party/drat-trim/	Official native LRAT checker pinned to a recorded commit
instances/	Canonical positive formula, negative exact signed-occurrence (3, 2, 2) gadget, and width-two MaxCut control
generated/	Four algebraic conformance benchmark packages

Path	Role
receipts/	Recorded test, exact-solver, analyzer, benchmark, and proof-verification outputs
PRIOR_ART_SEARCH_LOG.md	Dated query, collision, and stopping-boundary record

The implementation does not import the original TxGraffiti verifier. The canonical DIMACS instance was transcribed as an input object, then processed through separately implemented residual and graph code.

The threshold compiler's declared input domain is canonical set-valued CNF: complementary literals may coexist in a tautological clause, but the same literal token may not appear twice within one clause. Such duplicates are rejected explicitly instead of being silently normalized.

S3. Minimal reproduction

Run all commands from the artifact root.

```
python3 -m unittest discover -v
python3 -O -m unittest discover -v
c++ -std=c++17 -O2 -Wall -Wextra -pedantic bd_exact.cpp -o bd_exact
./bd_exact instances/txgraffiti_15_20.cnf
./bd_exact instances/negative_322.cnf
python3 analyze_formula.py instances/txgraffiti_15_20.cnf \
--solve --exact-solver ./bd_exact
make -C third_party/drat-trim lrat-check drat-trim
python3 verify_positive_base_proof.py \
--receipt receipts/positive-base-proof-verification.json
python3 verify_enforcer_composition_proof.py \
--receipt receipts/enforcer-composition-proof-verification.json
python3 verify_threshold_encoding_cleanroom.py \
instances/txgraffiti_15_20.cnf \
proof/positive-base-beta-le-0.cnf \
proof/positive-base-encoding-map.json \
--receipt receipts/threshold-encoding-cleanroom-validation.json
lean --run lean_terminal_signature.lean
python3 terminal_signature.py instances/sat2024_e322_enforcer.cnf \
receipts/sat2024-e322-enforcer-terminal.json \
--terminals 1 --verify-closed-beta
python3 generate_prism_amplifier.py 3 instances/prism_enforcer_s3.cnf \
--enforcer instances/sat2024_e322_enforcer.cnf \
--receipt receipts/prism-enforcer-s3.json
python3 generate_owest_amplifier.py 0 instances/owest_h1_e0.cnf \
--enforcer instances/sat2024_e322_enforcer.cnf \
--receipt receipts/owest-h1-e0.json
```

Expected high-level results:

Check	Expected result
Normal Python suite	25 tests, OK
Optimized Python suite	25 tests, OK
Canonical exact solve	14,348,907 partial; 939,975 bilateral; $\beta = 1$
Negative gadget	729 partial; 196 bilateral; $\beta = -1$
Canonical analyzer	degree 3; $\mu^* = 15$; $i = 16$; gap = 1
Positive-base certificate	both native LRAT checks verified; Lean direct LRAT verified; $\beta(P) = 1$
Enforcer-composition certificate	regenerated bytes; native and Lean LRAT verified; $\beta = 1$
Clean-room encoding audit	731-map bijection; all 9,256 clauses reconstructed exactly; 7-formula semantic corpus and mutation rejection pass
Connected-switch certificate	reconstructed formula; 2,686-variable encoding; native and Lean LRAT verified; $\beta = 1$
Enforcer terminal signature	Python: 4 feasible entries; Lean: exact four-row equality; closed $\beta(E) = 0$
Connected prism amplifier $s = 3$	72 variables; 96 clauses; graph order 240; $\beta = 3$
O--West amplifier $t = 0$	192 variables; 256 clauses; graph order 640; $\beta = 9$; ratio 67/64

The optimized run is essential because Python removes assert statements under -O. Agreement ensures that the tests do not pass only because a proof-relevant check was placed in an assertion in production code.

S4. Exact receipt semantics

bd_exact.cpp enumerates the ternary vectors in $\{0, 1, *\}^{V(F)}$. For each vector it:

1. classifies every indexed clause as satisfied or residual;
2. records unassigned literal polarities only from clauses already classified as residual;
3. checks both polarities for every unassigned variable;
4. computes $|T| - |U|$ for bilateral assignments;
5. updates the optimum, witness, full spectrum, and residual-size strata.

The canonical recorded receipt is receipts/txgraffiti-exact.json. Its full spectrum is:

```
{"1": 101540, "2": 455099, "3": 318306, "4": 61402,
 "5": 3597, "6": 31}
```

The minimum residual-clause counts for $u = 0, \dots, 15$ unassigned variables are:

```
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 14, 16, 18, 20
```

Subtracting u shows a minimum of one in every stratum. The returned witness 0000***** is an upper-bound certificate; the exhaustive strata are an execution receipt for the lower bound. Section S5 supplies the proof-system certificate.

The negative component receipt has witness 1***00 and spectrum

```
{"-1": 1, "0": 39, "1": 120, "2": 34, "3": 2}
```

The manuscript supplies a separate occurrence-count proof that excludes a value below -1 .

S5. Positive-base lower-bound certificate

The mathematical chain for the canonical formula P is:

1. the explicit witness `0000*****` is bilateral and has deficiency one;
2. the encoding lemma proves that `positive-base-beta-le-0.cnf` is satisfiable exactly when $\beta(P) \leq 0$;
3. the LRAT derivations prove that this encoding is unsatisfiable;
4. therefore $\beta(P) \geq 1$, and the witness gives equality.

The deterministic encoder uses one-hot ternary states for the 15 formula variables, exact residual indicators for the 20 indexed clauses, bilaterality implications, and a one-hot prefix-sum automaton for $\sum_a r_a - \sum_j u_j \leq 0$. The complete two-direction encoding proof is in `proof/README.md` and Appendix A of the manuscript.

The packaged encoding has 731 variables and 9,256 clauses. Reproduce all checks with:

```
python3 generate_positive_base_cnf.py \
  instances/txgraffiti_15_20.cnf \
  proof/positive-base-beta-le-0.cnf \
  --map proof/positive-base-encoding-map.json

make -C third_party/drat-trim lrat-check drat-trim

python3 verify_positive_base_proof.py \
  --receipt receipts/positive-base-proof-verification.json
```

The verifier regenerates the encoding and map in a temporary directory and requires byte equality with the package. It then checks the upper witness from the canonical DIMACS semantics, runs the pinned C checker on both the trimmed and direct LRAT proofs, and runs Lean 4 on the direct proof. The expected Lean transcript is:

```
c parsed 731 variables and 9256 clauses
c parsed 118497 LRAT actions
s LEAN_LRAT_VERIFIED
```

Critical SHA-256 values are:

```
d6b1d133018e4237fef73c3ed895a956e23f3d588dd92206254d49dfdc742813
instances/txgraffiti_15_20.cnf
c282d2d35f66e81ae175d9afa560b9a5ded2d00f51d8b02c380d939102156c11 proof/positive-base-beta-le-
0.cnf
b1dbad91b49076b1ed35d7ed2e5d3542d1c3bbb90f265692fb86a20242c508ee proof/positive-base-
encoding-map.json
b8fa3566bd8e1100281f7532ec7cf16918fe7329abbbb1ecd18920fcc3200370 proof/positive-base-
unsat.lrat
61bb30cadaf1cbf4bd15662a1a97b0a6352b416a187532c887000493f46f755a proof/positive-base-
direct.lrat
bf07c2ac96b9035da1ebcc578cb95e956a2b795629d613154cdb307f8a8f4a95 third_party/drat-trim/lrat-
check.c
```

The native checker is pinned to official `drat-trim` commit `2e3b2dc0ecf938addd779d42877b6ed69d9a985`. Lean's `LRAT.check_sound` theorem connects a successful checker result to `CNF.Unsat`; the wrapper materializes that implication in the successful branch. A standard-library clean-room validator independently reconstructs the 731-variable map and all 9,256 clauses, checks a seven-formula semantic corpus with an independent DPLL procedure, and rejects a rehashed clause mutation. The written general encoding lemma remains a mathematical translation obligation. This subsection records verified checking of one of the three finite UNSAT encodings, not full formalization of the manuscript or an unaffiliated reproduction.

S5.1 Attributed enforcer-composition certificate

The opposite-polarity composition of two published SAT 2024 enforcers is a second, structurally distinct 15-variable positive formula. Reproduce its byte regeneration, value-one witness, native LRAT check, and 35,737-action Lean check with:

```
python3 verify_enforcer_composition_proof.py \
  --receipt receipts/enforcer-composition-proof-verification.json
```

This formula is attributed synthesis from the published enforcer, not a new independently discovered base.

S5.2 Extended optional connected occurrence-switch certificate

The formula under `connectors/pp-beta1/` is reconstructed by swapping the recorded positive occurrence between two copies of P . The resulting 30-variable, 40-clause formula remains proper, simple, exact signed-occurrence $(3, 2, 2)$, and its formula graph is connected. Its recorded bilateral witness has deficiency one. The threshold compiler produces a 2,686-variable, 65,061-clause encoding of $\beta \leq 0$, and the direct LRAT proof refutes that encoding.

```
python3 verify_connected_switch_proof.py \
  --receipt receipts/connected-switch-proof-verification.json
```

No theorem in the manuscript depends on replaying this extended benchmark. The core package retains the formula, encoding, map, receipt and exact proof hash; the raw proof is a separately listed archival object of 5,793,599,477 bytes. Its expected Lean transcript is:

```
c parsed 2686 variables and 65061 clauses
c parsed 11393023 LRAT actions
s LEAN_LRAT_VERIFIED
```

The critical connected-proof hashes are:

```
83fe2bf036979a45aaea5f186feaa05a44eb67a1b44883d9c1fae9e4f2c947b3 connectors/pp-
beta1/connected-switch.cnf
54493aeb68ecd3d276cbf73765ef6c727275fc71e7a84ad195cdb1d0d8830c9d connectors/pp-beta1/beta-le-
0.cnf
84793f3c2e5773bee476342c182b598a1acb653b58905f7c09531ec8e026b823 connectors/pp-beta1/beta-le-
0-map.json
323402633cf890918c5690080279973a99fdd63fc80802d60842a958ba13db7f connectors/pp-beta1/beta-le-
0-direct.lrat
```

Native and Lean checking establish the same finite UNSAT subclaim through two checker implementations. Formula reconstruction and the written general threshold-encoding lemma remain the semantic bridge to $\beta = 1$.

S6. Cross-model test design

The strongest finite regression test is not a repeated optimum calculation. For each small indexed formula, `test_master_bijection_and_size_polynomial_exhaustively` enumerates:

- all bilateral ternary assignments and their deficiencies;
- all graph subsets that are independent dominating sets and their sizes;
- the forward and inverse maps of Theorem 2;
- the complete multiplicity identity $D_i(G(F), z) = z^k B_F(z)$.

The corpus contains more than one hundred formulas and includes:

- empty formulas and explicit unused variables;
- empty and unit clauses;
- binary clauses and tautologies;
- indexed duplicate clauses;
- formulas with satisfiable, unsatisfiable, positive, zero, and negative bilateral deficiency.

Other tests exhaustively compare width-two β with the MaxSAT defect, verify additivity and coefficient convolution, test width lifting between uniform clause widths, check MaxSAT recovery by replication, preserve duplicate indices through DIMACS, and recognize regular-DIM signatures. The workflow test mutates a generated graph and requires verification to reject the corrupted binding.

S7. Reproduce the algebraic conformance benchmarks

The checked packages already exist under generated/. Verify all four with:

```
for d in \
  generated/exact-gap-0 \
  generated/exact-gap-3 \
  generated/exact-gap-minus-2 \
  generated/general-minus-4
do
  python3 verify_benchmark.py "$d" --exact-solver ./bd_exact
done
```

Expected reported values are 0, 3, -2, -4, respectively. To regenerate an exact signed-occurrence (3, 2, 2) target, use:

```
python3 generate_benchmark.py \
  --family exact-322 \
  --target 3 \
  --output-dir generated/reproduced-gap-3
python3 verify_benchmark.py \
  generated/reproduced-gap-3 \
  --exact-solver ./bd_exact
```

For each package, verification performs all of the following:

1. validates the manifest schema;
2. checks the CNF and graph SHA-256 values;
3. recomputes the unique target recipe for the requested family;
4. separately solves each distinct component template;
5. reconstructs the entire variable-disjoint indexed formula;
6. compares the supplied graph byte-for-byte with deterministic formula-graph reconstruction;
7. applies $\beta(F_1 \sqcup F_2) = \beta(F_1) + \beta(F_2)$.

The graph hashes embedded in the manifests are:

Package	CNF SHA-256	Graph SHA-256
exact-gap-0	c7160ec0c5a5c4033218469b8666eb1a58650f0c0b633a45462bd2d6805aae56	2a191c5f7cb2325e75794232a1411a331830166996b9d067f6bf8
exact-gap-3	7387ed1faa53c038cc62b748d6de636cc0eb29e6daf17830f1106c6b36c50e43	48e2ea3192aec99ce1189467968f6357d19a13e060d8e2efc8c96
exact-gap-minus-2	af64e115a576767cb93ab9a974e9b2049880372fa06ada411b0fc25e7ecd4bf1	b3630a08bb1b2b8e68fe8579d480a879a2c9192f32b305afa2b8a
general-minus-4	ef6ade3361b3bb62b008d682cb38ecaf0a068b91c18f9f87cb04602fc32dd403	3cad14771b03310db4c9d364bf55531cbfab3ddbbf21cefdcaa3et

The exact-gap-minus-2 graph digest shown in its manifest and generated file is the authoritative value. A release-packaging integrity script should reject any mismatch between this table and the manifest; Section S9 lists the complete current file hashes and removes reliance on manual transcription.

S8. Development failure modes and corrections

An early C++ implementation recorded the polarity of an unassigned literal while scanning a clause, before knowing whether a later assigned literal satisfied the same clause. Such a clause must not contribute residual polarity. Cross-checking against the Python reference exposed the disagreement. The solver was changed to complete satisfaction classification before recording any residual signs.

During manuscript generation, a separate serialization boundary converted inline TeX escape sequences into control bytes. The source was recovered from the task's append-only patch record. A byte scan then confirmed zero unexpected controls and balanced inline and display delimiters. This packaging incident does not affect computations, but it motivates an explicit source-integrity gate before typesetting.

The first lower-bound encoder expressed the final cardinality inequality with a symmetric injection. The encoding was logically adequate but led CaDiCaL to produce more than 400 MB of an unfinished DRAT trace. That run was stopped; its partial proof was deleted and is not evidence. The retained transcript records the aborted attempt. Replacing the injection by a deterministic prefix-balance automaton yielded the current 9,256-clause encoding and a proof in seconds.

The LRAT file translated from the DRAT route passed the native checker but was not accepted by the Lean wrapper because of a format/ordering incompatibility. CaDiCaL's direct LRAT output is accepted by both checkers. Both successful native paths and the successful direct Lean path are retained; the failed Lean translation is development evidence only. A direct witness verifier also caught and corrected an early README transcription that said 12 unassigned variables and 13 residual clauses instead of the correct 11 and 12.

These failures are retained in the audit narrative because they demonstrate a general principle: agreement within one implementation is weaker than cross-representation checks, and rendered appearance is weaker than byte-level source validation.

S9. Core source and receipt hashes

The archive-level SHA256SUMS file is generated only after manuscript, supplement, receipts, and proof files are final. It is the normative complete hash inventory. Section S5 duplicates only the proof-critical values so that the lower-bound chain can be audited without scanning the whole manifest.

Test transcript hashes are non-normative because test durations can vary across runs. Their substantive pass condition is the reported test set and final OK.

S10. Independent reproduction checklist

- ☐ Confirm the canonical source archive or repository revision.
- ☐ Recompute the hashes rather than trusting this document.
- ☐ Run tests with and without Python optimization.
- ☐ Compile the C++ solver from source with warnings enabled.
- ☐ Compare the canonical and negative receipts with the expected search counts, values, spectra, and witnesses.
- ☐ Inspect at least one forward and inverse formula-graph witness manually.
- ☐ Regenerate the positive-base encoding and require byte identity.
- ☐ Run the clean-room encoding validator and require exact map/clause reconstruction, semantic-corpus agreement, and mutation rejection.
- ☐ Verify the compact LRAT files with the pinned native checker.
- ☐ Verify the two core direct LRAT files with Lean and require `LEAN_LRAT_VERIFIED`.
- ☐ Run `lean --run lean_terminal_signature.lean` and require `LEAN_TERMINAL_SIGNATURE_VERIFIED`.
- ☐ If reproducing the connected switch, verify its separate 5.79 GB LRAT with both native and Lean checkers and compare all four critical hashes.
- ☐ Read the encoding lemma; the proof file alone does not identify its SAT semantics.
- ☐ Regenerate one algebraic conformance benchmark in a fresh directory.
- ☐ Mutate one CNF or graph byte and confirm hash rejection.
- ☐ Treat receipts as records; identify the witness, LRAT derivation, and encoding lemma as the actual proof components.
- ☐ Review the manuscript proofs independently of all code.